# Raciocínio lógico e introdução ao Python

## Objetivo de aprendizagem

Modelar soluções com algoritmos, tipos, condições, repetição e validação de entradas usando Python.

## Teoria, aplicações e prática orientada

Esta apostila integra os 14 módulos de nivelamento M00 a M13. A grade superior de 10 semestres e 60 disciplinas permanece separada. Use a sequência de etapas disponível no site; este PDF também permite revisar temas anteriores.

## Como estudar

Leia os fundamentos, refaça os exemplos com a resposta escondida, resolva as questões e só então consulte o gabarito. Registre o motivo dos erros e retome a habilidade em outra sessão.

## Roteiro do material

1. Fundamentos
2. Fórmulas
3. Exemplos resolvidos
4. Aplicação
5. Exercícios
6. Gabarito
7. Aprofundamento aplicado e figura técnica
8. Problemas adicionais comentados
9. Videoaulas e bibliografia

Operações na calculadora do site: * para multiplicar, ^ para potência, sqrt(x) para raiz, log(x) para base 10 e ln(x) para base e. Escolha DEG ou RAD antes de usar trigonometria.

# 1. Fundamentos explicados

## 1.1 Problema, algoritmo e teste

Um algoritmo é uma sequência finita e precisa de instruções para resolver uma classe de problemas. Antes de codificar, descreva entradas, saídas e condições de erro. Um algoritmo correto precisa atender ao comportamento esperado não só em exemplos favoráveis, mas também em casos-limite.

## 1.2 Variáveis e tipos

Python dispõe de valores como int, float, bool e str. Atribuição com = guarda um valor em uma variável; comparação de igualdade usa ==. Valores de ponto flutuante têm limitações de representação, e comparar números decimais por igualdade exata pode falhar em determinados cálculos.

## 1.3 Decisões e operadores lógicos

Estruturas if/elif/else escolhem caminhos segundo expressões booleanas. Para uma divisão, trate denominador zero antes do cálculo. Operadores and, or e not combinam condições. Indentação não é detalhe visual em Python: faz parte da sintaxe.

## 1.4 Laços e funções

for percorre sequências; while repete enquanto uma condição for verdadeira, com atenção à terminação. Funções organizam regras reutilizáveis e facilitam testes: def corrente(v,r): ... . Prefira funções curtas com parâmetros claros e sem efeitos colaterais desnecessários.

## 1.5 Unidades, testes e saídas

Não misture volts, quilovolts ou miliampères sem converter. Teste casos normais, de fronteira e inválidos. Registre valores esperados e obtidos em tabela, e explique uma eventual diferença antes de considerar o código confiável.

# 2. Fórmulas e relações essenciais

1. Corrente: I = V/R, com R > 0 para o exemplo didático.

2. Conversão: I_mA = 1000 × I_A.

3. Estrutura: entrada → validação → cálculo → apresentação → teste.

### Atenção

Antes de substituir números numa fórmula, identifique as grandezas, padronize as unidades, verifique se o modelo se aplica e avalie se o resultado é plausível.

# 3. Exemplos resolvidos

### Exemplo 1: Função segura para Lei de Ohm

def corrente_mA(v, r): if r <= 0: raise ValueError("R deve ser positiva") return (v / r) * 1000 assert corrente_mA(9, 3000) == 3.0

### Exemplo 2: Decisão e repetição

valores = [2, 4, 6] soma = 0 for valor in valores: soma += valor media = soma / len(valores) # 4.0 Verifique o caso de lista vazia antes de fazer a divisão.

# 4. Aplicação prática

Faça um miniprograma de conversão de amperes para miliampères, com tratamento de erro para entradas não numéricas e três casos de teste anotados.

# 5. Exercícios de fixação

1. Qual operador testa igualdade em Python?
2. Qual o valor de 2**3?
3. O que ocorre se r=0 em v/r?
4. Qual o valor de soma após for i in [1,2,3]: soma += i, começando soma=0?
5. Por que 0.1+0.2 pode diferir de 0.3 em float?
6. Descreva três testes de uma função de corrente elétrica.

Tente resolver antes de avançar. Registre as unidades e escreva pelo menos uma frase explicando o método.

# 6. Gabarito comentado

Consulte apenas depois de tentar. A finalidade não é memorizar as linhas da resposta, e sim localizar o passo que precisa de revisão.

1. ==.

2. 8.

3. ZeroDivisionError, se não houver validação prévia.

4. 6.

5. Representação binária finita imprecisa de algumas frações decimais.

6. Ex.: 9 V/3000 Ω→3 mA; 12/2000→6 mA; R=0 deve ser rejeitada.

# 7. Aprofundamento aplicado

## Algoritmos que verificam entradas

Um algoritmo deve indicar quais dados recebe, quais condições aceita e qual saída produz. Em Python, input() retorna texto; float() converte representações numéricas adequadas e pode falhar. A validação precisa preceder a divisão para impedir um denominador nulo ou fisicamente inadmissível.

Use nomes de variáveis ligados à grandeza e à unidade, como tensao_v e resistencia_ohm. Python usa ponto em literais decimais; o operador ** representa potência, enquanto ^ tem outro significado na linguagem. Na calculadora do site, ^ foi definido como potência pelo interpretador próprio.

Funções ajudam a testar a mesma regra em vários casos. Condicionais escolhem o caminho; laços repetem operações. Separe o cálculo da leitura e apresentação para testar a função sem depender de digitação manual.

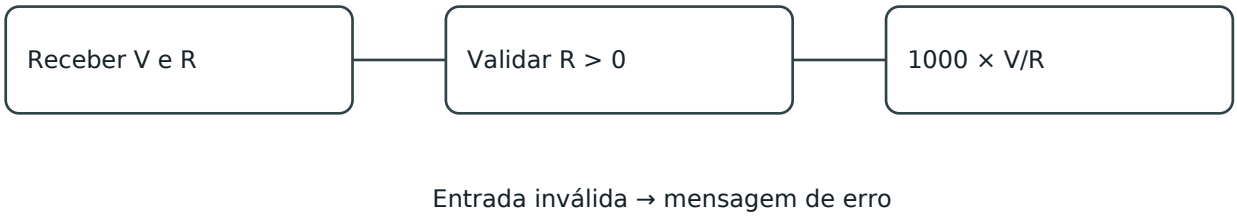


Figura didática original. Use as relações e hipóteses discutidas no texto.

### Exemplo adicional, passo a passo

Especifique uma função corrente_ma(tensao_v, resistencia_ohm) para um modelo resistivo.

1. Contrato: receber números finitos; exigir resistencia_ohm > 0. Uma tensão negativa pode representar a polaridade oposta da referência.

2. Se a resistência for inválida, produza um erro claro antes da divisão. Em Python, pode-se usar raise ValueError('Resistência deve ser positiva').

3. Calcule corrente_a = tensao_v / resistencia_ohm e retorne 1000 * corrente_a para obter mA.

4. Teste (9,3000) → 3; (12,2000) → 6; (-5,1000) → -5; (5,0) deve falhar. Inclua números não finitos na validação, usando math.isfinite().

## 8. Problemas adicionais

1. Por que a expressão '12' + '3' não calcula 15 em Python?

2. Qual é a sequência produzida por range(3) e quantas repetições há?

### Resolução comentada

1. Os dois operandos são strings; + concatena textos e produz '123'. Converta dados apropriados para número antes da soma; a conversão deve tratar entradas inválidas.

2. São os inteiros 0, 1 e 2; o limite final é excluído. Um laço que percorra essa sequência executa três vezes, salvo interrupção explícita.

## 9. Videoaulas complementares

### Curso Python #01 - Seja um Programador

Curso em Vídeo | Introdução à programação e ao Python

https://www.youtube.com/watch?v=S9uPNppGsGo

Seleção em português. Link e metadados públicos consultados em 12/11/2026. A disponibilidade futura e eventuais restrições de reprodução dependem do YouTube e do canal.

## Referências bibliográficas

Python Software Foundation. O tutorial do Python. Documentação oficial em português. Consulte números, fluxo de controle e exceções após a introdução à programação.

https://docs.python.org/pt-br/3/tutorial/

Textos e figuras deste material são explicações didáticas originais. As referências são indicadas para aprofundamento. Em montagens, use simulador ou fonte isolada de muito baixa tensão; não conecte os projetos à rede elétrica.